

SAPIENZA
UNIVERSITÀ DI ROMA

Integer Linear Programming

Volume 3 — MIP solvers

What a MIP solver does (presolve, cuts, primal heuristics, tree strategies, tolerances) and how to read its log, with real examples.

Fabio Furini

fabiofurini.github.io

Department of Computer, Control and Management Engineering — Sapienza University of Rome

Online version and downloadable material:
fabiofurini.github.io/integer-linear-programming/
github.com/fabiofurini/integer-linear-programming

Academic year 2026–2027

Contents

IV MIP solvers **2**

9 MIP solvers: what they do **3**

9.1 From the branch-and-cut algorithm to a MIP solver 4

9.2 The components of a MIP solver 5

9.3 How to read the log of Gurobi 7

9.4 Typical mistakes 13

9.5 In summary 13

Part IV

MIP solvers

Chapter 9

MIP solvers: what they do

Prerequisites

- From this module, the chapters «LP-based branch-and-bound algorithms» and «Branch-and-cut algorithms»: the branching, bounding and pruning operations, the cuts, and the incumbent solution.
- From this module, the chapter «LP relaxation and dual bounds»: the lower and upper bounds and the optimality gap.
- From the module *Algorithms*: the knapsack problem (KP) and the bin packing problem (BPP) are used as examples.

Objectives

At the end of this chapter you will be able to:

- name the components of a MIP solver and say on which side of the bounds each of them acts;
- explain presolve, cuts, primal heuristics, branching rules, tolerances and the gap;
- read the log of Gurobi and follow the two bounds, the incumbent solution and the tree;
- explain why a solver can stop with a solution that is not optimal.

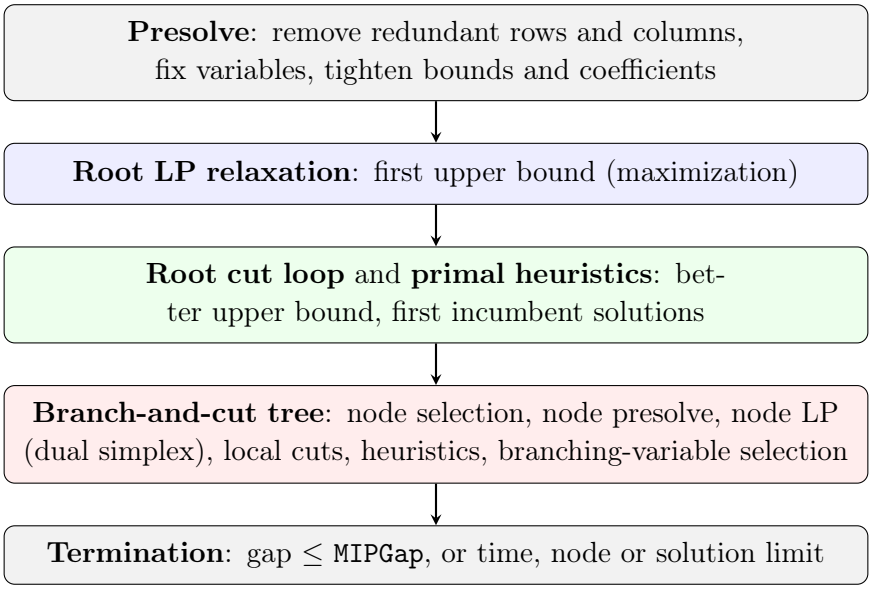
9.1 From the branch-and-cut algorithm to a MIP solver

- A **MIP solver** (Gurobi, CPLEX, Xpress, HiGHS, SCIP) is a computer program that computes optimal solutions of MIP problems. Its core is the branch-and-cut algorithm of the previous chapter.
- Around the core, the solver adds several components. Each of them acts on one side of the bounds on $z(\text{ILP})$, here for a maximization problem:

$$\underbrace{\text{LB}}_{\text{primal side}} \leq z(\text{ILP}) \leq \underbrace{\text{UB}}_{\text{dual side}}$$

- The components are the same in all the solvers. The details (which cuts, which heuristics, which rules) differ from solver to solver and from version to version, and are only partially documented.

A MIP solver executes presolve, then the root node (LP relaxation, cuts, primal heuristics), then the branch-and-cut tree, and it stops when the gap between the bounds is below a tolerance or a limit is reached.



9.1.1 Dictionary between the notes and the solver

In the notes (B&B, B&C)	In the Gurobi log
lower bound LB (max), incumbent solution $\bar{\boldsymbol{x}}$	`Incumbent`
upper bound UB (max)	`BestBd` (best bound)
optimality gap	`Gap`
list L of LP problems for branching	`Unexpl` (unexplored nodes)
LP problems already processed	`Expl` (explored nodes)
$z(\mathrm{LP}_k)$ of the current branching node	`Obj`
level of the node in the tree	`Depth`
fractional integer variables in $\tilde{\boldsymbol{x}}(\mathrm{LP}_k)$	`IntInf`

- In a minimization problem, such as the BPP, the roles are exchanged: `Incumbent` is an upper bound and `BestBd` is a lower bound.

9.2 The components of a MIP solver

9.2.1 Presolve

- The **presolve** transforms the problem into an equivalent one: same optimal value, and an optimal solution of the original problem can be recovered from one of the new problem.
- The new problem is smaller and, possibly, has a stronger LP relaxation. The typical operations are three:
  1. removing **redundant constraints** and **fixing variables** whose value is implied;
  2. **bound tightening**: deriving smaller upper bounds and larger lower bounds on the variables;
  3. **coefficient tightening**: changing the coefficients of a constraint so that the integer solutions stay the same, while the LP relaxation becomes smaller.

Example 9.1: coefficient tightening in the BPP model

Consider the BPP instance with bin capacity $c = 10$ and $n = 9$ items of weights

$$w_1 = \cdots = w_7 = 6, \qquad w_8 = w_9 = 3,$$

and the ILP model of the BPP with 9 bins ($9 \cdot 9 + 9 = 90$ binary variables, 18 constraints). The capacity constraint of bin b is

$$6\,(x_{1b} + \cdots + x_{7b}) + 3\,(x_{8b} + x_{9b}) \le 10\,y_b.$$

- Two items of weight 6 do not fit in the same bin ($6 + 6 > 10$), so $z(\mathrm{ILP}) = 7$. The LP relaxation has optimal value $\frac{7 \cdot 6 + 2 \cdot 3}{10} = \frac{24}{5}$.
- The presolve of Gurobi replaces the capacity constraint of every bin with

$$7\,(x_{1b} + \cdots + x_{7b}) + 3\,(x_{8b} + x_{9b}) \le 10\,y_b.$$

- The integer solutions are the same. If $y_b = 0$, all the x_{jb} are 0 in both constraints. If $y_b = 1$, a bin contains at most one item of weight 6 and at most one item of weight 3 ($7 + 3 = 10$), or no item of weight 6 and at most two items of weight 3 ($6 \leq 10$), in both constraints.
- The LP relaxation of the new model has optimal value $\frac{7 \cdot 7 + 2 \cdot 3}{10} = \frac{11}{2}$: the lower bound grows from $\frac{24}{5}$ to $\frac{11}{2}$ without changing the size of the model.

- The same reasoning can be applied at every branching node (**node presolve**, or **domain propagation**): a branching constraint $x_j \leq 0$ may imply that other variables can be fixed.

9.2.2 Cutting planes

- At the root node the solver executes several **rounds of cuts**. At the end, the log reports how many cuts of each family have been added.
- The families that appear in our logs are `Gomory` (Gomory cuts), `Cover` (cover inequalities) and `Zero half` (cuts with multipliers 0 and $\frac{1}{2}$, a family of Chvátal–Gomory cuts).
- Cuts are generated also in the tree, less often than at the root (branch-and-cut).

9.2.3 Primal heuristics

- **Primal heuristics** search for good integer feasible solutions: they give the incumbent solution and the bound LB (maximization). Examples are rounding the LP solution, **diving**, and local search around the incumbent solution (chapter «MIP heuristics»).
- In the log, a line starting with `H` reports a new incumbent solution found by a heuristic. A line starting with `*` reports one found as the integer optimal solution of a node LP problem (pruning by integrality).
- The line `Found heuristic solution` reports a solution found before the root LP.

9.2.4 Branching-variable and branching-node selection

- The branching-variable strategies of the B&B chapter (maximum infeasibility, strong branching, pseudocosts) are all available in a solver. Strong branching gives small trees but is expensive; pseudocosts are cheap but unreliable at the beginning.
- Solvers combine them in **reliability branching**: strong branching on a variable until its pseudocosts have been observed enough times.
- In Gurobi the parameter `VarBranch` selects the strategy: -1 automatic (default), 0 pseudo reduced-cost, 1 pseudo shadow-price, 2 maximum infeasibility, 3 strong branching.
- The node selection combines **best bound** (select the node with the largest $z(\mathrm{LP}_k)$, which decreases UB) and **depth first** (which finds integer solutions quickly).

9.2.5 Tolerances and gap

- A MIP solver works with **floating-point arithmetic**: a value is integer, a constraint is satisfied, a reduced cost is non-positive, up to a **tolerance**. The default values in Gurobi 13 are:

Parameter	Default	Meaning
`IntFeasTol`	10^{-5}	a variable is integer if it is within 10^{-5} of an integer
`FeasibilityTol`	10^{-6}	maximum violation of a constraint
`OptimalityTol`	10^{-6}	tolerance on the reduced costs (LP optimality)
`MIPGap`	10^{-4}	relative gap at which the solver stops
`MIPGapAbs`	10^{-10}	absolute gap at which the solver stops

- Gurobi computes the **relative gap** as

$$\text{Gap} = \frac{|\,\texttt{BestBd} - \texttt{Incumbent}\,|}{|\,\texttt{Incumbent}\,|},$$

  which, for a maximization problem, is the optimality gap $\frac{\text{UB}-\text{LB}}{|\text{LB}|}$ of the notes, with UB not rounded down.

"`Optimal solution found (tolerance 1.00e-04)`" means that the incumbent solution is optimal **up to a relative gap of** 10^{-4}. With a larger `MIPGap` the solver stops with a solution that may **not** be optimal, and it writes the same sentence.

9.3 How to read the log of Gurobi

- All the logs are produced by Gurobi 13.0.3 (`gurobipy`) with `Threads` $= 1$ and `Seed` $= 0$, so that the execution is deterministic. With other versions or computers the numbers of nodes and iterations can differ.
- The numbers on the left are line numbers added for the comments; the lines `Set parameter ...` are omitted.

9.3.1 A knapsack instance solved by the presolve

Example 9.2: a KP instance

Capacity $c = 9$ and $n = 6$ items:

	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$
p_j	6	5	8	9	6	7
w_j	2	3	6	7	5	9

The model has 6 binary variables and 1 constraint. The LP relaxation has optimal value $z(\text{LP}) = 6 + 5 + \frac{4}{6} \cdot 8 = \frac{49}{3}$; the optimal solution is $\{1, 4\}$, of value 15.

Gurobi log: KP instance, default parameters

```
01| Gurobi Optimizer version 13.0.3 build v13.0.3rc0 (linux64 - "Ubuntu 26.04 LTS")
02|
03| CPU model: Intel(R) Core(TM) i5-14500T, instruction set [SSE2|AVX|AVX2]
04| Thread count: 20 physical cores, 20 logical processors, using up to 1 threads
05|
06| Non-default parameters:
07| LogToConsole  0
08| Threads  1
09|
10| Optimize a model with 1 rows, 6 columns and 6 nonzeros (Max)
11| Model fingerprint: 0xb4d2031c
12| Model has 6 linear objective coefficients
13| Variable types: 0 continuous, 6 integer (6 binary)
14| Coefficient statistics:
15|   Matrix range     [2e+00, 9e+00]
16|   Objective range  [5e+00, 9e+00]
17|   Bounds range     [1e+00, 1e+00]
18|   RHS range        [9e+00, 9e+00]
19|
20| Found heuristic solution: objective 11.0000000
21| Presolve removed 1 rows and 6 columns
22| Presolve time: 0.00s
23| Presolve: All rows and columns removed
24|
25| Explored 0 nodes (0 simplex iterations) in 0.00 seconds (0.00 work units)
26| Thread count was 1 (of 20 available processors)
27|
28| Solution count 2: 15 11
29|
30| Optimal solution found (tolerance 1.00e-04)
31| Best objective 1.500000000000e+01, best bound 1.500000000000e+01, gap 0.0000%
```

01–04 Version of Gurobi, computer, threads used (1, since `Threads` = 1).

06–08 Parameters with non-default values; `Seed` = 0 is not listed, since 0 is its default.

10–13 Size of the model: 1 row, 6 columns, 6 non-zeros, `(Max)`; the variable types (6 binary).

14–18 **Coefficient statistics**: the smallest and the largest absolute values of the coefficients. Ranges spanning many orders of magnitude signal numerical difficulties; here all values are between 2 and 9.

20 A heuristic finds a solution of value 11 before the presolve, so LB = 11.

21–23 The presolve removes all rows and columns: the problem is solved without LP relaxation and without branching.

25 0 nodes, 0 simplex iterations.

28–31 Two solutions found, of values 15 and 11; the best has value 15, the best bound is 15 and the gap is 0: the solution is optimal.

9.3.2 A bin packing instance solved at the root node

Gurobi log: BPP instance of the presolve example, default parameters

```
01| Gurobi Optimizer version 13.0.3 build v13.0.3rc0 (linux64 - "Ubuntu 26.04 LTS")
02|
03| CPU model: Intel(R) Core(TM) i5-14500T, instruction set [SSE2|AVX|AVX2]
04| Thread count: 20 physical cores, 20 logical processors, using up to 1 threads
05|
06| Non-default parameters:
07| LogToConsole  0
08| Threads  1
09|
10| Optimize a model with 18 rows, 90 columns and 171 nonzeros (Min)
11| Model fingerprint: 0x05ef7999
12| Model has 9 linear objective coefficients
13| Variable types: 0 continuous, 90 integer (90 binary)
14| Coefficient statistics:
15|   Matrix range     [1e+00, 1e+01]
16|   Objective range  [1e+00, 1e+00]
17|   Bounds range     [1e+00, 1e+00]
18|   RHS range        [1e+00, 1e+00]
19|
20| Found heuristic solution: objective 7.0000000
21| Presolve time: 0.00s
22| Presolved: 18 rows, 90 columns, 171 nonzeros
23| Variable types: 0 continuous, 90 integer (90 binary)
24|
25| Root relaxation: objective 5.500000e+00, 28 iterations, 0.00 seconds (0.00 work units)
26|
27|     Nodes    |    Current Node    |     Objective Bounds      |     Work
28|  Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time
29|
30|      0     0    5.50000    0    7    7.00000    5.50000  21.4%
31|      0     0    5.50000    0    9    7.00000    5.50000  21.4%
32|      0     0 infeasible    0         7.00000    7.00000  0.00%
33|
34| Cutting planes:
35|   Gomory: 2
36|   Cover: 4
37|   Zero half: 1
38|
39| Explored 1 nodes (56 simplex iterations) in 0.00 seconds (0.00 work units)
40| Thread count was 1 (of 20 available processors)
41|
42| Solution count 1: 7
43|
44| Optimal solution found (tolerance 1.00e-04)
45| Best objective 7.000000000000e+00, best bound 7.000000000000e+00, gap 0.0000%
```

10–13 18 rows, 90 columns, 171 non-zeros; `(Min)`: minimization.

20 A heuristic finds a solution with 7 bins. In a minimization problem 7 is an **upper bound**.

21–23 The presolved model has the same size, but the coefficients 6 have become 7 (Example 9.1).

25 **Root relaxation**: the value is $\frac{11}{2}$ (lower bound), after 28 simplex iterations.

27–28 Header of the **node log**: `Expl` and `Unexpl` are the explored and unexplored nodes; `Obj`, `Depth`, `IntInf` describe the current node; `Incumbent`, `BestBd`, `Gap` are the two bounds and the gap.

30 Root node: 7 integer variables are fractional, and the gap is $\frac{7-5.5}{7} \approx 21.4\%$.

31 After a round of cuts the LP value is still $\frac{11}{2}$, with 9 fractional variables.

32 After more cuts the LP becomes `infeasible`: no solution with fewer than 7 bins exists. The best bound jumps to 7, the gap to 0, and the root is pruned.

34–37 Cuts added: 2 Gomory, 4 cover, 1 zero-half.

39–45 1 explored node, 56 simplex iterations; one solution, with 7 bins, optimal.

Here the heuristic finds the optimal solution at once. All the remaining work, presolve and cuts, serves to **prove** optimality, by raising the lower bound from $\frac{24}{5}$ to 7.

9.3.3 The same instance without presolve, cuts and heuristics

- With `Presolve` $= 0$, `Cuts` $= 0$ and `Heuristics` $= 0$ the solver is close to the LP-based B&B algorithm of the notes, and we see the tree.

Gurobi log: BPP instance, `Presolve=0`, `Cuts=0`, `Heuristics=0`

```
01| Gurobi Optimizer version 13.0.3 build v13.0.3rc0 (linux64 - "Ubuntu 26.04 LTS")
02|
03| CPU model: Intel(R) Core(TM) i5-14500T, instruction set [SSE2|AVX|AVX2]
04| Thread count: 20 physical cores, 20 logical processors, using up to 1 threads
05|
06| Non-default parameters:
07| Heuristics  0
08| Cuts  0
09| LogToConsole  0
10| Presolve  0
11| Threads  1
12|
13| Optimize a model with 18 rows, 90 columns and 171 nonzeros (Min)
14| Model fingerprint: 0x05ef7999
15| Model has 9 linear objective coefficients
16| Variable types: 0 continuous, 90 integer (90 binary)
17| Coefficient statistics:
18|   Matrix range     [1e+00, 1e+01]
19|   Objective range  [1e+00, 1e+00]
20|   Bounds range     [1e+00, 1e+00]
21|   RHS range        [1e+00, 1e+00]
22|
23| Variable types: 0 continuous, 90 integer (90 binary)
24|
25| Root relaxation: objective 4.800000e+00, 31 iterations, 0.00 seconds (0.00 work units)
26|
27|     Nodes    |    Current Node    |     Objective Bounds      |     Work
28|  Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time
29|
30|      0     0    4.80000    0    8          -    4.80000      -     -    0s
```

```
31|      0     0    4.80000    0    8          -    4.80000      -     -    0s
32|      0     2    4.80000    0    8          -    4.80000      -     -    0s
33| *   38    36              38       7.0000000    4.80000  31.4%
34|
35| Explored 1211 nodes (3669 simplex iterations) in 0.04 seconds (0.03 work units)
36| Thread count was 1 (of 20 available processors)
37|
38| Solution count 1: 7
39|
40| Optimal solution found (tolerance 1.00e-04)
41| Best objective 7.000000000000e+00, best bound 7.000000000000e+00, gap 0.0000%
```

25 Without presolve the root relaxation has value $\frac{24}{5}$, the value of the original model.

30–32 No incumbent solution yet (-). At line 32 the root is branched and 2 children wait in the list.

33 After 38 nodes a node LP, at depth 38, has an integer optimal solution with 7 bins (*). The gap is $\frac{7-4.8}{7} \approx 31.4\%$.

35 To prove that 7 is optimal the solver explores 1211 nodes (3669 iterations), against 1 node with the default parameters.

9.3.4 The gap as a stopping criterion

- For a KP with $n = 30$ items (strongly correlated: $w_j \in \{10, \dots, 100\}$, $p_j = w_j + 10$, $c = 846$), the default run is solved by the presolve, with $z(\text{ILP}) = 1036$. We switch off the three components and set `MIPGap` $= 0.01$:

Gurobi log: KP with $n = 30$, `Presolve=0`, `Cuts=0`, `Heuristics=0`, `MIPGap=0.01`

```
01| Gurobi Optimizer version 13.0.3 build v13.0.3rc0 (linux64 - "Ubuntu 26.04 LTS")
02|
03| CPU model: Intel(R) Core(TM) i5-14500T, instruction set [SSE2|AVX|AVX2]
04| Thread count: 20 physical cores, 20 logical processors, using up to 1 threads
05|
06| Non-default parameters:
07| MIPGap  0.01
08| Heuristics  0
09| Cuts  0
10| LogToConsole  0
11| Presolve  0
12| Threads  1
13|
14| Optimize a model with 1 rows, 30 columns and 30 nonzeros (Max)
15| Model fingerprint: 0x713eb478
16| Model has 30 linear objective coefficients
17| Variable types: 0 continuous, 30 integer (30 binary)
18| Coefficient statistics:
19|   Matrix range     [2e+01, 1e+02]
20|   Objective range  [2e+01, 1e+02]
21|   Bounds range     [1e+00, 1e+00]
22|   RHS range        [8e+02, 8e+02]
23|
24| Variable types: 0 continuous, 30 integer (30 binary)
25|
26| Root relaxation: objective 1.044592e+03, 1 iterations, 0.00 seconds (0.00 work units)
27|
28|     Nodes    |    Current Node    |     Objective Bounds      |     Work
29|  Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time
```

```
30|
31|      0     0 1044.59155    0    1          - 1044.59155      -     -    0s
32|      0     0 1044.57143    0    1          - 1044.57143      -     -    0s
33|      0     2 1044.57143    0    1          - 1044.57143      -     -    0s
34| *   21    20              20    1032.0000000 1044.41270  1.20%
35| *   61    39              20    1033.0000000 1044.24324  1.09%
36| *   84    53              22    1035.0000000 1044.00000  0.87%
37|
38| Explored 85 nodes (83 simplex iterations) in 0.00 seconds (0.00 work units)
39| Thread count was 1 (of 20 available processors)
40|
41| Solution count 3: 1035 1033 1032
42|
43| Optimal solution found (tolerance 1.00e-02)
44| Best objective 1.035000000000e+03, best bound 1.044000000000e+03, gap 0.8696%
```

26 The root relaxation needs 1 simplex iteration: as in the Dantzig bound, only the split item is fractional (`IntInf` $= 1$).

34–36 Three incumbent solutions, all integer node LP solutions (`*`): 1032, 1033, 1035. The best bound decreases from 1044.41270 to 1044 and the gap from 1.20% to 0.87%.

38–44 The solver stops after 85 nodes, since $\frac{1044-1035}{1035} \approx 0.87\% < 1\%$, and it writes "`Optimal solution found (tolerance 1.00e-02)`" for the value 1035.

- The optimal value is 1036: the bound 1044 certifies only that 1035 is within 0.87% of the optimum. With the default `MIPGap`, the same run explores 3339 nodes to close the gap.

9.3.5 Branching-variable selection

- With presolve, cuts and heuristics off, we change only `VarBranch`. The table gives the explored nodes (simplex iterations in parentheses).

`VarBranch`	KP, $n = 30$	BPP, $n = 9$
-1 automatic	3339 (3388)	1211 (3669)
0 pseudo reduced cost	3339 (3388)	1499 (3877)
1 pseudo shadow price	3346 (3392)	5009 (14846)
2 maximum infeasibility	3341 (3390)	69404 (116946)
3 strong branching	3341 (3390)	61192 (105887)

- For the KP the choice is almost irrelevant: every node LP solution has at most one fractional variable, the split item, so there is nothing to choose.
- For the BPP the number of nodes changes by a factor larger than 50. The model is highly **symmetric**: the 9 bins are identical, so every solution appears many times with the bins renamed.
- On such models the behaviour of the rules is hard to predict; here the automatic choice is the best.

Reading a log means following the two bounds: the `Incumbent` column is the primal side (heuristics and integer nodes), the `BestBd` column the dual side (relaxation, presolve, cuts,

branching).

9.4 Typical mistakes

Warning

- **Reading "Optimal solution found" as a proof of optimality.** The sentence reports the tolerance in use: with `MIPGap` $= 0.01$ the solution can be 1% away from the optimum.
- **Swapping the bounds in a minimization problem.** For the BPP, `Incumbent` is the upper bound and `BestBd` the lower bound.
- **Believing that the tree explains everything.** With the default parameters, most of the work of the solver is outside the tree: in our logs the presolve alone solves the KP.
- **Comparing node counts across runs.** The numbers depend on the version of the solver, on the computer and on the seed; with other parameters the same instance can need 1 or 61192 nodes.
- **Expecting the solver to follow the rules of the notes.** The choices of the solver (cuts, heuristics, node selection) are not documented in detail, and they change with the version.

9.5 In summary

In summary

- A MIP solver is a branch-and-cut algorithm with presolve, cuts, primal heuristics, branching and node-selection rules around it.
- Presolve and cuts raise the lower bound (minimization), heuristics find the incumbent solution, branching closes the gap.
- The log shows the bounds: `Incumbent` is the primal side, `BestBd` the dual side, `Gap` their relative distance.
- The solver stops when the gap is below `MIPGap`: the solution is optimal only up to that tolerance.
- Switching off the components shows how much each of them saves on a given instance.

Dashboard of the BPP instance. Lower and upper bounds on $z(\text{ILP}) = 7$ in the logs of this chapter:

Stage	Lower bound	Upper bound
LP relaxation of the original model	$\frac{24}{5}$	—
Heuristic solution	$\frac{24}{5}$	7
LP relaxation after presolve	$\frac{11}{2}$	7
Root node, after the cuts	7	7